

Урок 15. Корутины. Обработка исключений. Async, suspend.

05 июня 2020

В этом уроке разбираемся к чему приводят исключения в async корутине и в suspend функциях

async

В двух прошлых уроках мы разбирали обработку исключений в корутинах launch. Корутины async в этом плане похожи. Исключение также будет передаваться в родительскую корутину и далее вверх по иерархии корутин и отменять все на своем пути. CoroutineExceptionHandler и SupervisorJob могут быть использованы для обработки исключений и изменения поведения отмены.

Основное отличие в том, что у async корутины есть метод await для получения результата работы этой корутины.

Рассмотрим простой пример, где есть родительский launch, а в его дочернем async выбрасывается исключение:

```
1 launch {
2
3     val deferred = async {
4         // exception
5     }
6
7     // ...
8
9     val result = deferred.await()
10
11    // ...
12
13 }
```

launch запускает async корутину и методом await подписывается на ожидание результата. Когда в async произойдет исключение, оно будет поймано и отправлено в родительский launch и далее вверх. Т.е. родительский launch будет отменен. А метод await вместо результата выбросит то самое исключение, которое произошло в async.

Вопрос: надо ли в этом случае помещать await в try-catch и **ловить** это исключение? Давайте рассуждать.

Ошибка уже ушла в самую верхнюю корутину, по пути все поотменяв. Оттуда она будет отправлена на обработку. Это либо в CoroutineExceptionHandler (если он есть), либо в глобальный обработчик (т.е. крэш). Это произойдет независимо от того, обернули мы вызов метода await в try-catch или нет.

Но try-catch сможет повлиять на выполнение кода launch до конца.

Если мы **не ловим** исключение из await в try-catch, то launch сам его ловит и завершает свою работу. Он снова отправит ошибку наверх, но это уже ничего не изменит. А код, который в launch был расположен после вызова await, не будет выполнен.

Если же мы **ловим** исключение - то launch продолжает выполнять свой код, который находится после await. Напомню, что launch уже отменен и должен проверять isActive, чтобы понять, надо ли ему продолжать свою работу. Но даже если он отменен, после await может располагаться какой-то важный код, который надо выполнить в любом случае. Например - закрытие ресурсов или соединений. Или сохранение куда-либо пометки о том, что работа не была завершена нормально.

Таким образом, в примере с родительским launch оборачивать await в try-catch имеет смысл, если вам необходимо, чтобы код после await был выполнен.

SupervisorJob

Отдельно надо рассмотреть случай, когда родителем async является не обычный launch, а SupervisorJob.

В конце прошлого урока мы рассматривали, как это можно сделать с помощью небольшого костыля:

```
1 launch {
2
3     val deferred = async(SupervisorJob(coroutineContext[Job])) {
4         // exception
5     }
6
7     // ...
8
9     val result = deferred.await()
10
11    // ...
12
13 }
```

Напомню, что в этом случае вместо связи:

```
launchJob > asyncJob
```

мы получаем:

```
launchJob > supervisorJob > asyncJob
```

Это приводит к тому, что ошибка из async не пойдет вверх по иерархии корутин, т.к. supervisorJob ее туда не пропустит. При этом он скажет async корутине, что не может обработать ошибку. Это означает, что async корутине надо сделать это самостоятельно. Но она (в отличие от launch) не умеет отправлять ошибку в обработчики. Она просто сохраняет это исключение внутри себя, чтобы выбросить его, когда вызовут ее метод await.

"Счастливым", вызвавшим метод `await`, в нашем примере является `launch`. До вызова `await` у него все было хорошо. Он не был отменен дочерним асупс (в котором произошла ошибка), потому что `supervisorJob` не позволил этого сделать. Т.е. корутина `launch` работает в штатном режиме и ей есть что терять. И если не обернуть `await` в `try-catch`, то получится так, что внутри `launch` произойдет исключение. Оно отменит `launch` и пойдет дальше вверх по корутинам все отменять.

Поэтому в случае, когда мы от асупс отгораживаемся с помощью `SupervisorJob`, ловить ошибку из `await` надо обязательно.

Случаи с `coroutineScope` и `supervisorScope` рассмотрим в отдельном уроке.

suspend

Рассмотрим, как корутина реагирует на ошибки, происходящие в `suspend` функции. Это зависит от того, как сама `suspend` функция обработала ошибку. Есть несколько вариантов.

resumeWithException

Напомню, что в `suspend` функции нам доступен `continuation`, у которого есть не только `resume` метод для возврата в корутину результата работы, но и `resumeWithException` для возврата ошибки.

Если, например, `suspend` функция пыталась скачать файл, но получила или поймала ошибку, она эту ошибку может передать в `resumeWithException`. Для корутины это будет равносильно тому, что вызов `suspend` функции выбросил исключение. А это мы уже рассматривали в прошлых уроках.

Если вызов `suspend` функции был обернут в `try-catch`, то ошибка никуда дальше не пойдет. Если `try-catch` не было, то корутина прерывается и ошибка идет вверх по иерархии корутин.

Теперь давайте рассмотрим варианты, когда ошибку в `suspend` функции не удалось поймать. В этом случае результат зависит от того, где именно произошла ошибка. Есть два варианта.

Синхронная часть

В уроке про `suspend` функции мы разбирали, что эти функции должны свою работу выполнять асинхронно, в отдельном потоке, чтобы не блокировать вызывающий их поток. Но если результат работы готов сразу, то его можно вернуть не запуская фоновую работу.

Рассмотрим пример:

```
1 suspend fun someFunction(): String =
2     suspendCancellableCoroutine { continuation ->
3
4         val result = getFromCache()
5
6         if (result != null) {
7             continuation.resume(result)
8         } else {
9             // async code
10        }
11    }
12 }
```

Есть какая-то `suspend` функция, которая перед тем, как начать асинхронную работу, проверяет кэш. Если она находит там результат, то сразу отправляет его обратно в корутину.

Если этот код выбросит исключение, то это будет равносильно тому, что `suspend` функция в корутине выбросила исключение.

Асинхронная часть

Если же ошибка произойдет в асинхронной части и не будет там поймана в `try-catch`, то приложение свалится с крэшем. И даже если обернуть `suspend` функцию в `try-catch` или использовать `CoroutineExceptionHandler`, это не поможет.

Потому что это был отдельный поток. И выполнялась там не корутина, которая автоматически ловит все ошибки своего кода, а обычный код. А непойманное исключение в обычном коде приводит к крэшу.